

**Je sais coder, comment je
partage mon code pour le lancer
ailleurs que sur mon poste ?**

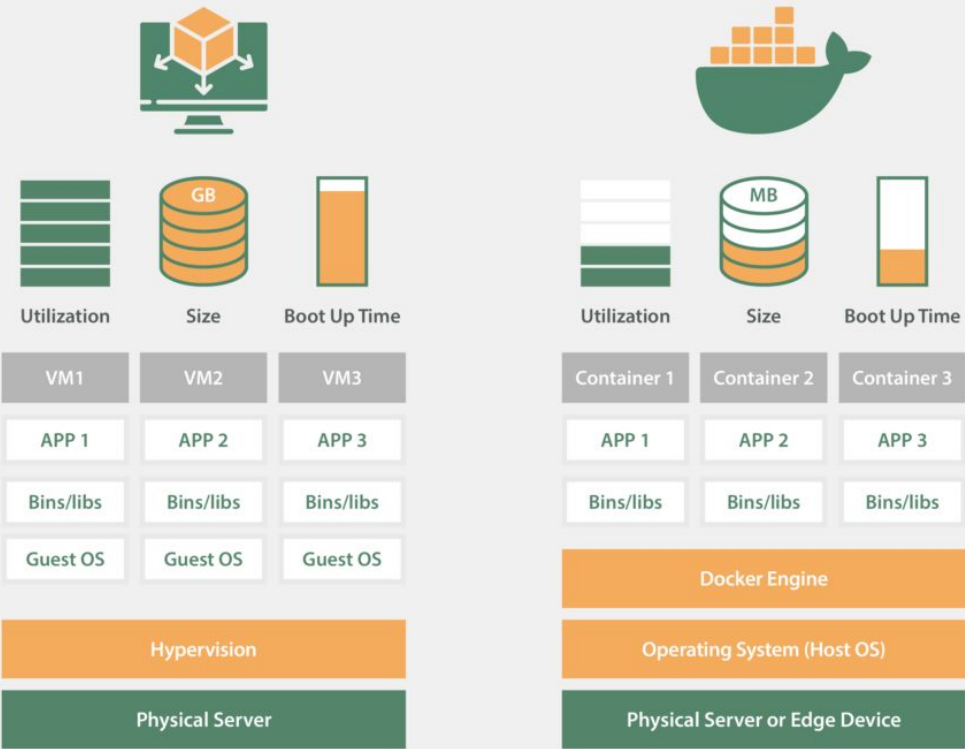
Conteneurs & images



Pourquoi on se pose cette question ?

Quick quiz VM vs Conteneur

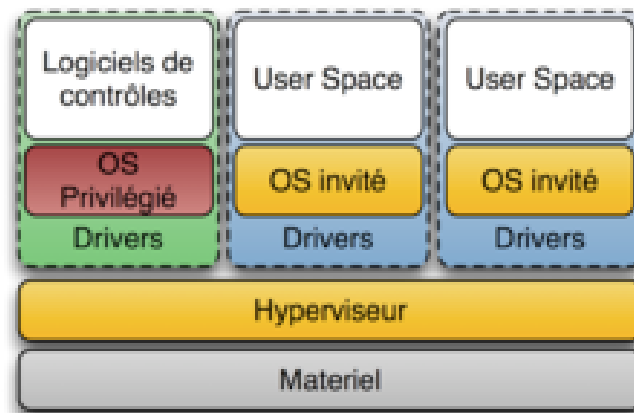
Virtual Machine vs. Docker



Virtualisation

Hyperviseur type 1: Bare-metal

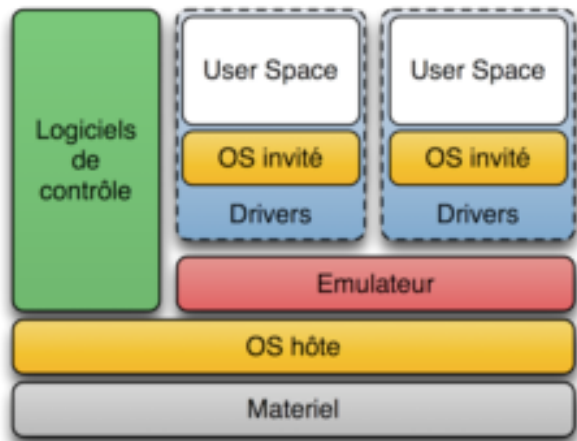
- On utilise un serveur sur lequel on installe un noyau léger dédié à gérer plusieurs systèmes d'exploitation (OS)
- On a donc peu d'overhead et on peut gérer plusieurs machines virtuelles (VMs)
- On a une très bonne isolation des VMs
- On a de très bonnes performances et on profite au mieux du matériel (hardware)



Virtualisation

Hyperviseur type 2: Hosted

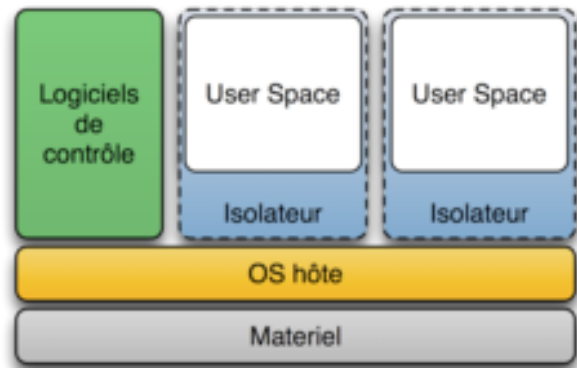
- On utilise un serveur avec un OS puis à l'aide d'un émulateur on peut gérer plusieurs systèmes d'exploitation (OS)
- On a plus d'overhead et on peut gérer plusieurs machines virtuelles (VMs)
- Très simple et pratique, fonctionnalité de snapshot, très facile de connecter les VMs entre elles ou de monter des disques partagés
- L'émulateur (software) gère l'isolation



Virtualisation

Conteneur:

- On utilise un serveur avec un OS puis avec des process noyau on peut gérer plusieurs conteneurs.
- On partage le même noyau donc ça va très vite
- Très simple et pratique, très facile de connecter les VMs entre elles ou de monter des disques partagés
- Le noyau doit gérer l'isolation



Virtualisation

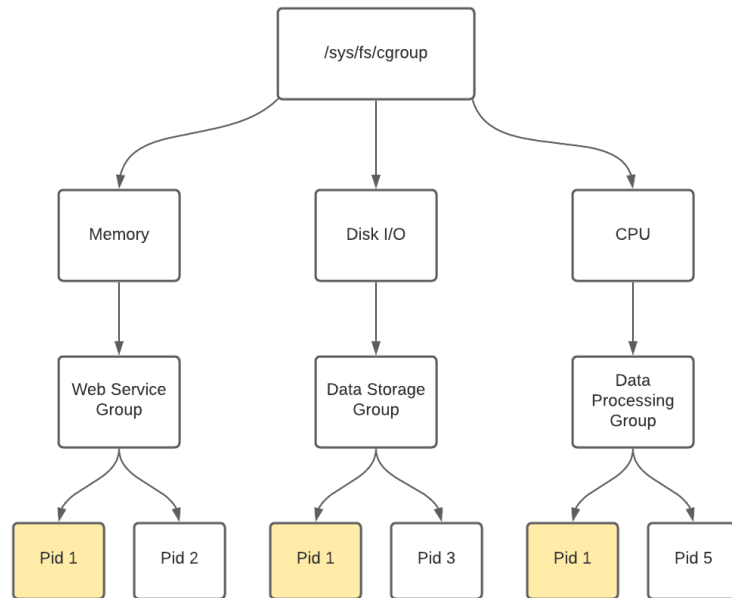
Conteneur avec Linux - Namespace:

- Le noyau peut gérer plusieurs “userspace” qui servent de conteneurs
- On utilise les “namespaces” du noyau qui permettent d’isoler ce qu’un process peut “voir”
- Sous linux, il existe 8 types de “namespace”: PID, NET, MNT, IPC, UTS, USER, CGROUP, TIME
- Donc un conteneur correspond à un process isolé dans un “userspace” (composé de 8 “namespace”)
- Un “namespace” peut-être partagé entre plusieurs userspace (et donc plusieurs conteneurs)

Virtualisation

Conteneur avec Linux – Control Group:

- Le noyau peut gérer plusieurs “userspace” qui servent de conteneurs
- On utilise les “cgroups” du noyau qui permettent de définir des limites de ressources à un process ou un groupe de process
- Les “cgroups” permettent également de prioriser l’allocation de ressources



Virtualisation

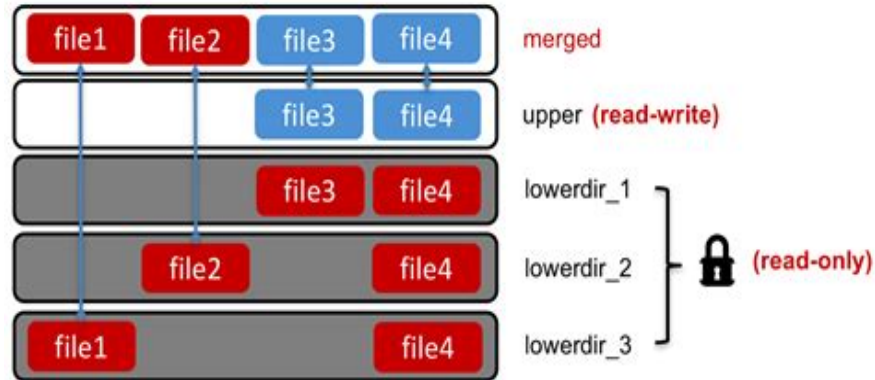
Conteneur avec Linux - Overlay:

- Pour gérer les systèmes de fichiers des différents conteneurs, le noyau va construire plusieurs couches de fichiers.
- Le “lowerdir” est en read-only et contient toute la base immuable du système (les binaires systèmes, les bibliothèques, ...)
- Le “upperdir” est la partie “writable” du conteneur, il contient toutes les fichiers que le conteneur modifie
- Le “mergedir” = lower + upper et la partie visible du conteneur.

Virtualisation

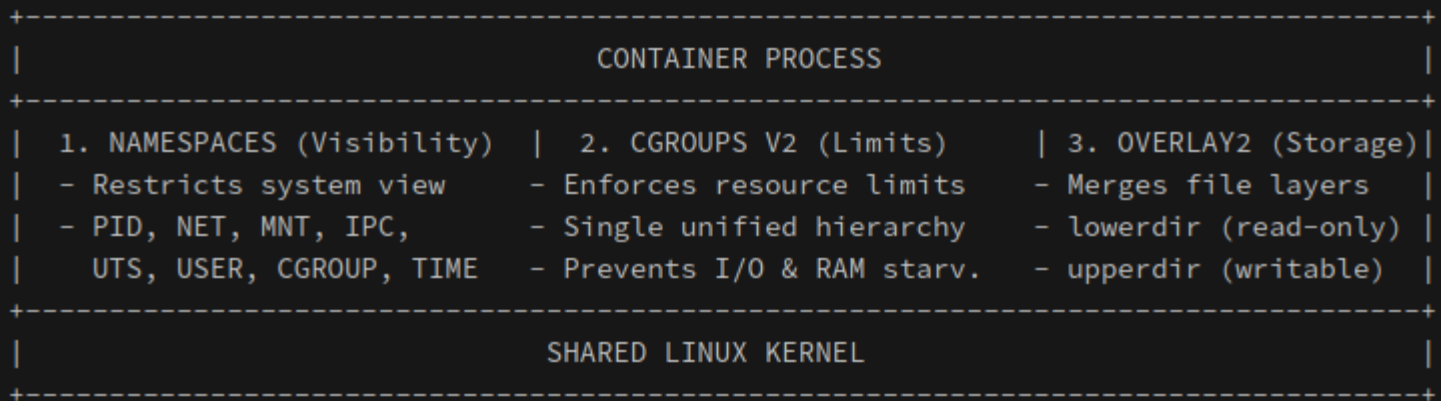
Conteneur avec Linux - Overlay:

- Il est très facile de partager le “lower” (qui peut correspondre à plusieurs “lowerdir” empilés)
- On peut réécrire juste ce dont on a besoin dans le “upper”
- Permet d’optimiser l’espace disque pour tous les conteneurs
- C’est beaucoup plus rapide que les disques virtuels des VMs

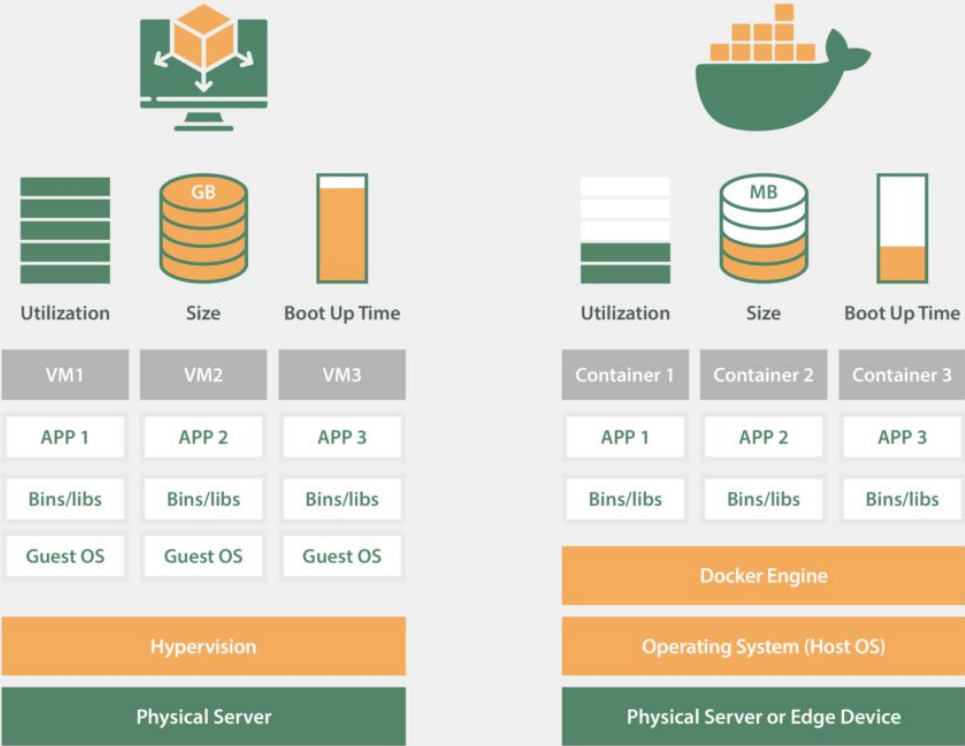


Virtualisation

Conteneur avec Linux:



Virtual Machine vs. Docker



“Ça marche chez moi”

“Ça marche chez moi”

- dépendances
- OS
- versions
- ports
- secrets.
- ...



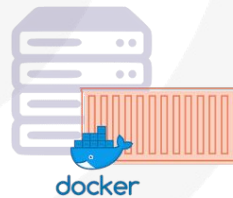
local status: OK



server status: ERROR
E: library XYZ missing



local status: OK



server status: OK

Static website

NGINX + modsecurity + openssl +
bootstrap

User DB

PostgreSQL + PLV8 +
V8

Analytics DB

Hadoop + Hive + Thrift +
OpenJDK

Cache

Redis + Redis
Sentinel

Background workers

Python 3.5 + celery + pyredis +
libcurl
+ ffmpeg + libopencsv + nodejs +
phantomjs

Web frontend

Ruby + Rails + SASS +
Unicorn

API endpoint

Python 2.7 + flask + pyredis
+ celery
+ psycop +
postgresql-client

Development VM

Production cloud

Disaster recovery

QA server

Contributor's laptop

Customer appliances

Static website	?	?	?	?	?	?	?
Web frontend	?	?	?	?	?	?	?
Background workers	?	?	?	?	?	?	?
User DB	?	?	?	?	?	?	?
Analytics DB	?	?	?	?	?	?	?
Cache	?	?	?	?	?	?	?
	Development VM	Customer appliances	QA server	Disaster recovery	API endpoint	Contributor's laptop	Production cloud

Une problématique rencontrée dans l'industrie

Des biens variés

peluches, tonneaux,
violons, ordinateurs,
lampes, céréales...

Des infrastructures variées

bateaux, camions,
grues, warehouse...

	?	?	?	?	?	?	?
	?	?	?	?	?	?	?
	?	?	?	?	?	?	?
	?	?	?	?	?	?	?
	?	?	?	?	?	?	?
	?	?	?	?	?	?	?
							

Format standard

pour envoyer des biens



Standard d'API

pour livrer du code

Static website							
Web frontend							
Background workers							
User DB							
Analytics DB							
Cache							
	Development VM	Customer appliances	QA server	Disaster recovery	API endpoint	Contributor's laptop	Production cloud

Container images

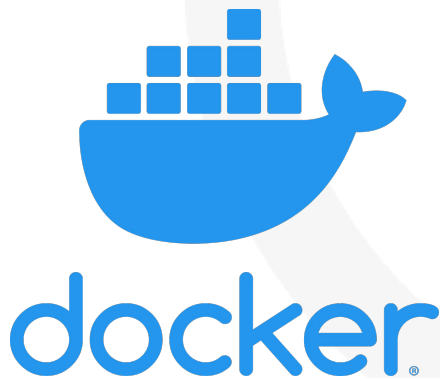
un standard d'API - les conteneurs



OPEN CONTAINER
INITIATIVE

Container runtimes

faire tourner des conteneurs dans des environnements isolés



podman

Docker

CLI :

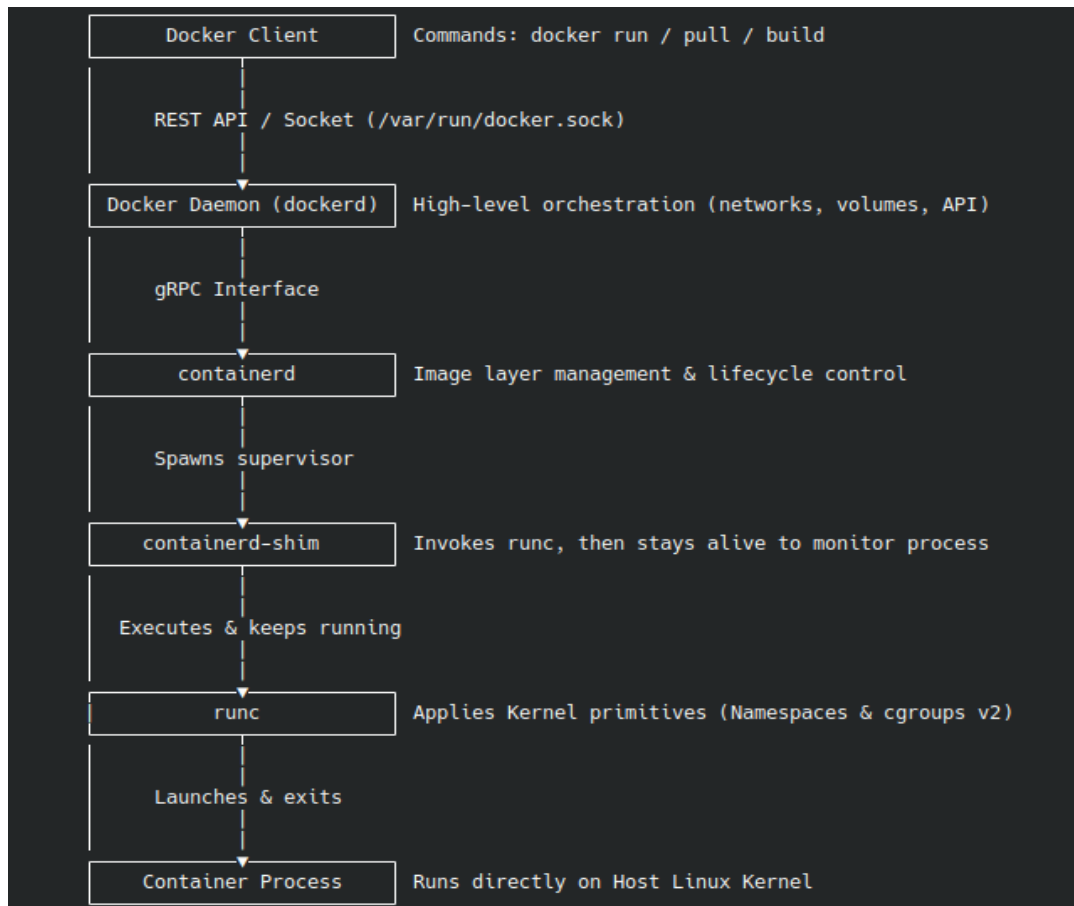
- La CLI permet d'interagir avec le "daemon" dockerd via des appels API
- On peut faire des commandes comme "docker build", "docker pull", "docker push", "docker image ls", "docker image rm", "docker tag", "docker history", "docker save", "docker commit" pour gérer les images
- On peut faire des commandes comme "docker ps -a", "docker run", "docker start", "docker stop", "docker restart", "docker exec", "docker logs", "docker inspect", "docker stats", "docker top", "docker pause" pour gérer les containers
- Il existe beaucoup d'autres sous-commandes comme "docker volume", "docker network", "docker system", "docker login", ...

Docker

Daemon :

- Le “daemon” permet de traduire vos appels d’API en action concrète sur le système. On peut donc créer des volumes persistants, des images et des réseaux
- Il interagit avec “containerd” qui appelle “containerd-shim” qui appelle “runc”
- “containerd” est un runtime open source de conteneur, il gère le “haut niveau” de la gestion d’un conteneur: gérer les layer, les images, les tar files,...
- “containerd-shim” est un process léger qui reste actif afin de suivre les activités des conteneurs sans le daemon (restart ou upgrade) et il appelle “runc”
- “runc” est un runtime léger de conteneur, il gère le “bas niveau” de la gestion d’un conteneur en appelant les fonctions kernel: gérer les namespaces, gérer les cgroups, lancer le premier process

Docker

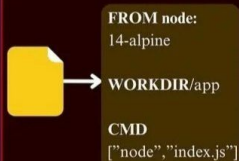


Docker Concepts Explained

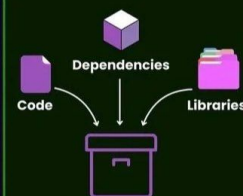
Coding Snow



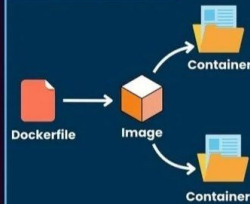
Docker file



Docker Image



Docker container



Docker Registry

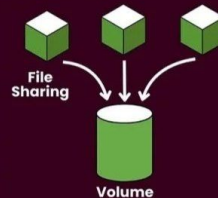


"But it works on my machine"
"Ok, we'll ship your machine"

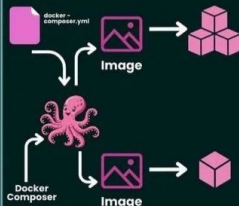


And that's how
Docker was born!!

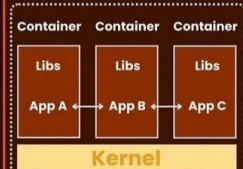
Docker Image



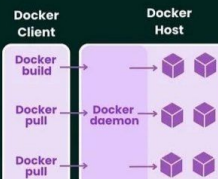
Docker Image



Docker Network



Docker CLI



Conteneurs

Anatomie d'une image

nginx/nginx-ingress:latest
825db0262774

Layers (43)

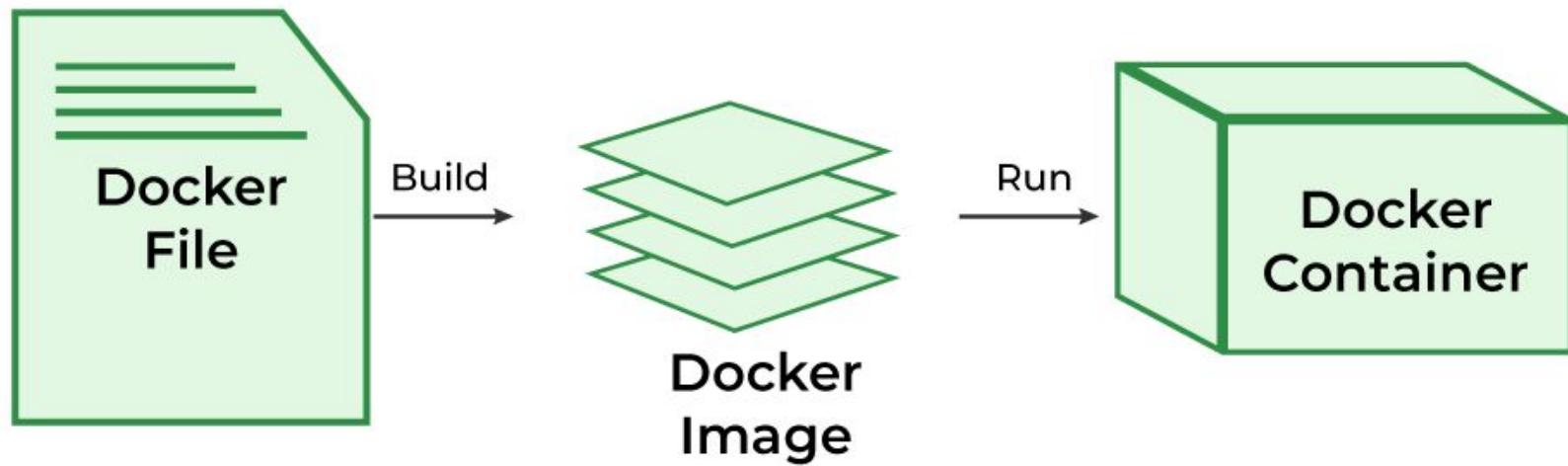
0	# debian.sh --arch 'arm64' out/ 'bookworm' '@1757289600'	107.73 MB
1	LABEL maintainer=NGINX Docker Maintainers <docker-maint@nginx.com>	0 B
2	ENV NGINX_VERSION=1.29.1	0 B
3	ENV NJS_VERSION=0.9.1	0 B
4	ENV NJS_RELEASE=1~bookworm	0 B
5	ENV PKG_RELEASE=1~bookworm	0 B
6	ENV DYNPKG_RELEASE=1~bookworm	0 B
7	RUN /bin/sh -c set -x && groupadd --system --gid 101 nginx && useradd --system --gid nginx --no-cr...	104.71 MB
8	COPY docker-entrypoint.sh / # buildkit	8.19 KB
9	COPY 10-listen-on-ipv6-by-default.sh /docker-entrypoint.d # buildkit	12.29 KB
10	COPY 15-local-resolvers.envsh /docker-entrypoint.d # buildkit	12.29 KB
11	COPY 20-envsubst-on-templates.sh /docker-entrypoint.d # buildkit	12.29 KB
12	COPY 30-tune-worker-processes.sh /docker-entrypoint.d # buildkit	16.38 KB
13	ENTRYPOINT ["/docker-entrypoint.sh"]	0 B
14	EXPOSE map[80/tcp:{}]	0 B
15	STOPSIGNAL SIGQUIT	0 B
16	CMD ["nginx" "-g" "daemon off;"]	0 B

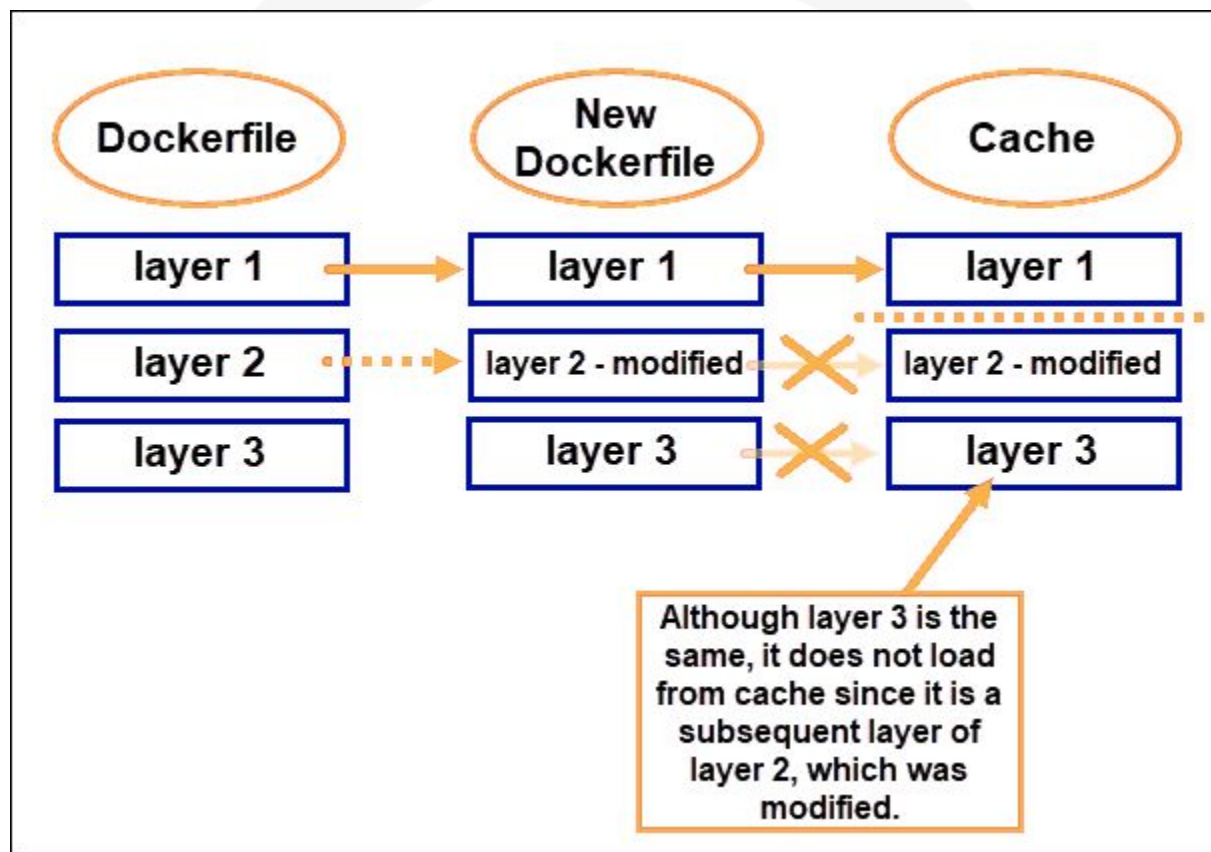
Base Image

Variables

Files added

Entry point (run command)





Dockerfiles

recette d'une image pour conteneur

Start from an image that already contains Python 3.8.

FROM python:3.8

Install dependencies; be explicit about versions.

RUN pip install flask==1.1

Copy the application's source code into the image.

COPY src/ /code

Add useful information for operators.

EXPOSE 80/tcp

Specify how to start the application.

ENTRYPOINT python /code/main.py



Dockerfiles

Un bon Dockerfile

syntax=docker/dockerfile:1

Stage 1 : build

FROM golang:1.23-alpine AS builder

Dossier de travail

WORKDIR /src

Copie du code source

COPY . .

Compile le binaire

RUN go build -o hello .

Stage 2 : runtime

FROM alpine:3.20 AS runtime

Copie uniquement le binaire depuis le builder

COPY --from=builder /src/hello /usr/local/bin/hello

Commande par défaut

ENTRYPOINT ["hello"]

Bonnes pratiques

- multi-stage
- cache (BuildKit)
- deps verrouillées
- user non-root
- bases minimales
- SBOM
- scans

Pièges

- Tag latest
- UID/GID
- permissions volumes
- secrets dans l'image



Conteneurs

Du build local à la registry

version sémantique que tu veux publier

VERSION=1.0.0

ton registry Harbor (exemple : harbor.monentreprise.com)

REGISTRY=harbor.monentreprise.com

ton projet Harbor (namespace)

PROJECT=demo

nom de l'image

IMAGE=hello-multi

docker build -t \${REGISTRY}/\${PROJECT}/\${IMAGE}:\${VERSION}

docker inspect --format='{{index .RepoDigests 0}}'

\${REGISTRY}/\${PROJECT}/\${IMAGE}:\${VERSION}

harbor.monentreprise.com/demo/hello-multi@sha256:abcd1234ef567890...

Faire tourner un conteneur

Nginx qui écoute sur 8080 dans le conteneur → mappé sur 8080 local.

docker run --rm -d -p 8080:8080 nginx:alpine

monter un répertoire local (./html) dans /usr/share/nginx/html du conteneur

**docker run --rm -d -p 8080:8080 -v \$(pwd)/html:/usr/share/nginx/html:ro **
nginx:alpine

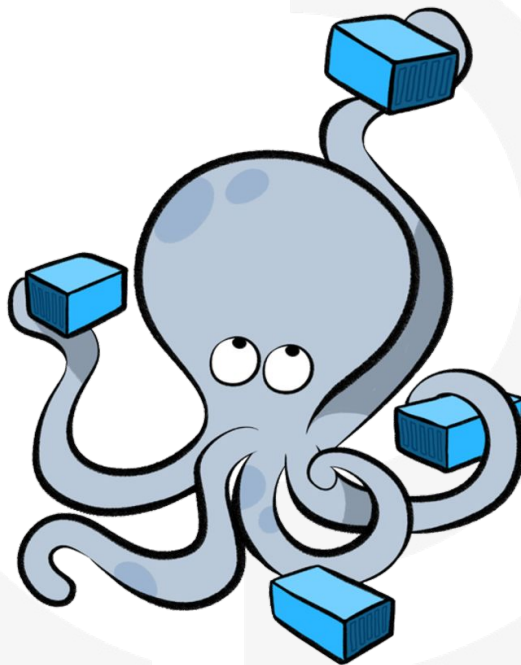
une appli qui lit APP_ENV et SECRET_KEY

**docker run --rm -it **
**-e APP_ENV=production **
**-e SECRET_KEY=super-secret-value **
myapp:1.0.0

Faire tourner plusieurs conteneurs

nécessite du service discovery

Docker compose



version: "3.9"

services:

frontend:

image: nginx:alpine

container_name: frontend

ports:

- "8080:80"

volumes:

On monte un dossier local "frontend" contenant index.html, etc.

- ./frontend:/usr/share/nginx/html:ro

depends_on:

- backend

backend:

build: ./backend

un Dockerfile Flask dans ./backend

container_name: backend

environment:

- DATABASE_URL=postgresql://user:password@db:5432/appdb

volumes:

On monte le code source local pour développement

- ./backend:/app

ports:

- "5000:5000"

depends_on:

- db

db:

image: postgres:16-alpine

container_name: db

environment:

POSTGRES_USER: user

POSTGRES_PASSWORD: password

POSTGRES_DB: appdb

volumes:

Persistance des données PostgreSQL

- db_data:/var/lib/postgresql/data

ports:

- "5432:5432"

volumes:

db_data:

```
.
├── docker-compose.yml
├── frontend/
│   └── index.html
├── backend/
│   ├── Dockerfile
│   └── app.py
```

Destination la Production

nécessite du service discovery

En dev

👉 Objectif : outils + hot reload + debug

👉 Caractéristiques : tooling complet, volumes montés, root autorisé, taille moins critique.

- Monte le code source local avec
-v \$(pwd):/app
- Peut être root (pratique pour debug)
- Beaucoup plus lourd (pleins d'outils)

En prod

👉 Objectif : minimale, performante, non-root

👉 Caractéristiques : multi-stage, pas d'outils superflus, user dédié.

- Basée sur distroless
(mince, pas de shell)
- Pas de root
- Pas d'outils inutiles
→ surface d'attaque réduite

On construit l'image pour toutes les architectures

**docker buildx build **

**--platform linux/amd64,linux/arm64 **

**-t myregistry.com/project/myapp:1.0.0 **

**--push **

.

On vérifie le digest

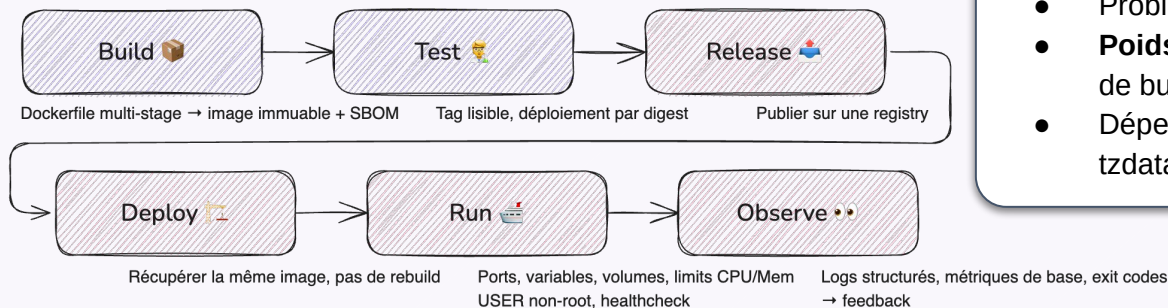
docker buildx imagetools inspect

myregistry.com/project/myapp:1.0.0

CONTENEURS & IMAGES - ONE PAGER

Pourquoi (les enjeux)

- Uniformiser les déploiements
- Simplifier les configurations



Clés

- **Immutabilité** (digest) • **Portabilité** • **Reproductibilité**
- **Sécurité by design** : non-root, bases minimales, secrets hors image
- **Partage d'équipe** : Compose pour dev, registry pour prod, GitOps-ready

 **Promesse** : Du code à un artefact immuable qu'on lance ailleurs sans surprises

Pièges à éviter

- Tag **latest** en prod • **retag** sauvage • rebuild silencieux
- Problèmes **UID/GID** et permissions de volumes
- **Poids d'image** inutilement gros (caches, artefacts de build)
- Dépendances natives manquantes (openssl, tzdata, locales)

Checklist

- .dockerignore • **multi-stage** • lockfiles (deps figées)
- Images **minces** (distroless/alpine/ubi-minimal)
- **Pas de secrets** dans l'image (env/secret store)
- Versionner & publier **par digest** (+ release notes)
- compose.yaml clair (healthcheck, réseaux, overrides)

Q & A